Car Rental System

Here is the Project Context and the database Class Diagram.

Read and analyze them carefully to understand the project, architecture, roles, rules, workflows, and database structure.

Do not write or modify any code.
Do not start implementing anything.
Just understand the project context and wait for my next instruction.

Car Rental System

I want to build a complete car rental system consisting of two main parts:

• **Admin Part**: A Web Dashboard dedicated to administration.
• **Client Part**: Available as a website and a mobile application, allowing the customer to browse cars, book them, and track their reservations.

The system relies on **Authentication and Authorization**, with two main **Roles**: Admin and Client.

1. Admin Part

The system provides the administration with a set of main functionalities:

Dashboard

The dashboard provides a set of statistics and general information.

Manage Customers & Staff

The Admin is responsible for managing customers and staff, where they can:

- Add new customers and Admin.
- View customer and staff records.

Manage Reservations

The Admin is responsible for managing reservations and tracking the lifecycle of each reservation.

For customers who wish to rent a car without using the website or application, the Admin can create an account for them and then register the reservation on their behalf.

The Admin can also:

• Create a new reservation.
• Cancel or delete cancelled reservations.
• Change the reservation status and track the rental stages.

The reservation lifecycle is, for example:

Pending → Confirmed → Active → Completed

Where:

• **Pending**: The reservation request has been created and is still waiting for confirmation.
• **Confirmed**: The reservation has been approved.
• **Active**: The car rental process has started and the car is now rented to the

customer.
• **Completed**: The rental period has ended and the car has been returned.

Manage Cars

The Admin is responsible for managing cars, where they can:

• Add a new car.
• Edit car information.
• Delete a car using **Soft Delete**.
• Track the status of each car.
• See the cars available or unavailable for rental.
• See the cars currently rented.
• we use claudinary for register image of each cars

2. Client Part

The Client can use the system through the **website or mobile application**.

The customer can:

- Browse the available cars.
- View car information, such as the brand, model, price, images, and specifications.
- Select the rental start and end dates.
- See the cars available during the specified period.
- Select the appropriate car and create a reservation.

However, in order for the customer to make a reservation, they must first:

- Create a new account (**Register**), or
- Log in (**Login**) if they already have an account.

Reservation Process

The customer first selects the **rental start and end dates**, after which the system checks the existing reservations and displays the cars available during that period.

After selecting a car, the customer submits a reservation request, and the reservation status is initially:

Pending

After the administration reviews and approves it, it becomes:

Confirmed

When the rental process begins, it

becomes:

Active

When the car is returned and the rental process ends, it becomes:

Completed

Reservation Tracking

The customer can access their account and track all their reservations and view the status of each reservation, such as:

- Pending
- Confirmed
- Active
- Completed

They can also view the details of each

reservation, such as the car, rental start date, end date, and total price.

3.Technologies & Architecture

Frontend:

- **Web:**
  1. Next.js with React and javascript.
  2. React Icons for icons
  3. Axios for HTTP requests
  4. TanStack React Query for server-state management and API data fetching

- **Mobile Application:**
  1. React Native with Expo
  2. React Navigation for navigation, including:
     -Stack Navigator
     -Bottom Tab Navigator

-Drawer Navigator

1. React Native Vector Icons for icons
2. Axios for HTTP requests

Backend:

1. Spring Boot with Java
2. Spring Security for authentication and authorization
3. JWT (JSON Web Token) for token-based authentication
4. Spring Data JPA for database access and persistence
5. MySQL as the database
6. XAMPP as the local development server/database environment
7. Backend architecture follows a layered structure:
   -Model
   -DTO

-Repository
-Service
-Controller

4. Security

The system must include:

- Authentication using **JWT**.
- Password encryption.
- API protection.
- Role-based authorization.

Thus, the system provides an integrated experience for managing the car rental process, starting from managing cars, customers, and reservations by the Admin, to browsing cars, making reservations, and tracking reservation

status by the Client.